

Guía práctica: conexión a una cámara mediante RTSP

Configuración, pruebas y acceso desde Python/OpenCV para cámaras IP y sistemas de videovigilancia.

1. ¿Qué es RTSP?

RTSP (Real Time Streaming Protocol) es un protocolo utilizado para controlar y recibir transmisiones de video en tiempo real desde cámaras IP, DVR, NVR y otros dispositivos. RTSP normalmente se utiliza junto con protocolos de transporte como RTP/RTCP.

Una dirección RTSP suele tener esta estructura:

```
rtsp://USUARIO:CONTRASEÑA@IP_CAMARA:PUERTO/RUTA_DEL_STREAM
```

2. Datos que necesitas de la cámara

Dato	Ejemplo	Descripción
Dirección IP	192.168.1.100	IP local de la cámara
Puerto RTSP	554	Puerto RTSP habitual
Usuario	admin	Usuario configurado en la cámara
Contraseña	*****	Credencial de acceso
Ruta del stream	/stream1	Depende del fabricante/modelo

3. Ejemplo de URL RTSP

```
rtsp://admin:MiClave123@192.168.1.100:554/stream1
```

Importante: la ruta exacta depende del fabricante. Algunas cámaras usan rutas diferentes para el stream principal y el stream secundario. Consulta el manual de tu modelo si la URL anterior no funciona.

4. Verificar conectividad de red

Antes de intentar abrir el video, comprueba que el computador pueda comunicarse con la cámara. En Windows puedes utilizar:

```
ping 192.168.1.100
```

Para comprobar específicamente el puerto RTSP puedes utilizar PowerShell:

```
Test-NetConnection 192.168.1.100 -Port 554
```

Si el puerto aparece como accesible, existe conectividad TCP hacia ese puerto. Esto no garantiza por sí solo que la URL RTSP sea correcta.

5. Probar la cámara con VLC

Una forma sencilla de descartar problemas de código es probar primero el stream con **VLC Media Player**. En VLC selecciona **Medio** → **Abrir ubicación de red** y pega la URL RTSP.

```
rtsp://usuario:contraseña@192.168.1.100:554/ruta
```

Si VLC reproduce correctamente el video, puedes utilizar la misma URL desde Python, OpenCV, FFmpeg u otras herramientas.

6. Conectarse desde Python y OpenCV

Instala OpenCV si todavía no lo tienes:

```
pip install opencv-python
```

Ejemplo básico para visualizar el stream:

```
import cv2

rtsp_url = "rtsp://usuario:contraseña@192.168.1.100:554/ruta"

cap = cv2.VideoCapture(rtsp_url)

if not cap.isOpened():
    print("No fue posible conectar con la cámara.")
    raise SystemExit

while True:
    ret, frame = cap.read()

    if not ret:
        print("No se pudo recibir un frame.")
        break

    cv2.imshow("Camara RTSP", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

7. Integrarlo con YOLO

Si tu objetivo es utilizar la cámara con detección de objetos, puedes reemplazar la visualización directa por una inferencia YOLO sobre cada frame.

```
from ultralytics import YOLO
import cv2

model = YOLO("best.pt")

rtsp_url = "rtsp://usuario:contraseña@192.168.1.100:554/ruta"
cap = cv2.VideoCapture(rtsp_url)

while cap.isOpened():
    ret, frame = cap.read()

    if not ret:
        break

    results = model(frame)

    annotated = results[0].plot()
```

```

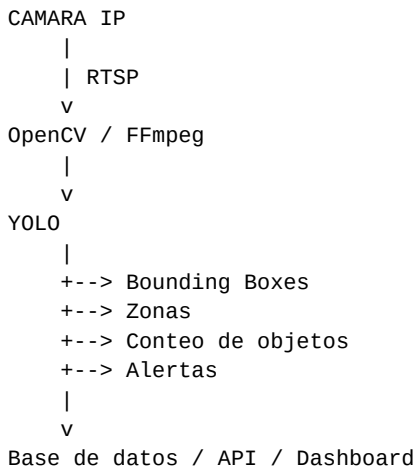
cv2.imshow("YOLO + RTSP", annotated)

if cv2.waitKey(1) & 0xFF == ord("q"):
    break

cap.release()
cv2.destroyAllWindows()

```

8. Arquitectura recomendada



9. Problemas frecuentes

Problema	Posible causa / solución
No conecta	Verifica IP, puerto, usuario, contraseña y conectividad de red.
VLC tampoco conecta	Probablemente la URL RTSP, credenciales o configuración de la cámara son incorrectas.
VLC funciona pero OpenCV no	Puede existir un problema con el backend de video/FFmpeg utilizado por OpenCV.
Video entrecortado	Revisa ancho de banda, resolución, FPS, codec y carga del computador.
Demasiado retraso	Reduce resolución/FPS o utiliza FFmpeg/GStreamer con una configuración de baja latencia.
Credenciales con caracteres especiales	Caracteres como @, ., / o # pueden requerir URL encoding.

10. Recomendaciones para un proyecto real

- **No guardes las credenciales directamente en el código.** Utiliza variables de entorno o un archivo de configuración que no se suba a GitHub.
- Usa un stream secundario si necesitas ejecutar YOLO con menor consumo de CPU/GPU.
- Configura reconexión automática si la cámara pierde temporalmente la conexión.
- Registra logs para saber cuándo se perdió el stream y cuándo se restableció.
- Para múltiples cámaras, considera separar captura, inferencia y almacenamiento en componentes independientes.
- Si el sistema debe funcionar durante largos períodos, evalúa FFmpeg o GStreamer para una canalización de video más robusta.

11. Ejemplo de configuración mediante variables de entorno

```
RTSP_HOST=192.168.1.100
RTSP_PORT=554
RTSP_USER=admin
RTSP_PASSWORD=MíClave
RTSP_PATH=/stream1
```

En Python puedes construir la URL a partir de esa configuración. Esto facilita cambiar la cámara sin modificar el código principal y evita publicar credenciales accidentalmente.

12. Flujo de implementación recomendado

1. Configurar la cámara y habilitar RTSP.
2. Identificar IP, puerto y ruta del stream.
3. Probar la URL con VLC.
4. Verificar la conexión desde Python/OpenCV.
5. Agregar reconexión y manejo de errores.
6. Integrar YOLO sobre los frames.
7. Agregar zonas y reglas de detección.
8. Guardar eventos, imágenes o métricas según las necesidades del sistema.
9. Exponer resultados mediante API o dashboard si el proyecto lo requiere.

13. Nota de seguridad

Evita exponer directamente el puerto RTSP de una cámara a Internet. Cuando sea posible, mantén las cámaras en una red privada/VLAN y utiliza mecanismos seguros de acceso remoto. Protege especialmente las credenciales de administración y las URLs que las contienen.